Syntax Checker Project Report
For a Simple Programming Language

Group:
Iyana Taylor - 2209566
Kenrick Brown - 2209589
Devonic McDonald - 2101569

University of Technology, Jamaica
Faculty of Engineering and Computing
School of Computing and Information Technology

CIT3006

Tutor: Khalilah Burrell-Battick
April 10, 2026

Table of Contents

Grammar Design and Justification

Terminals: ^
( )
*
/
+
-
variable
number

Variables or Non-Terminals:
S
A
B
C

Start symbol: S

Final Grammar:
S → S + A | S - A | A
A → A * B | A / B | B
B → C ^ B | C
C → (S) | variable | number

Rationale

The grammar has four (4) levels associated with four (4) distinct arithmetic levels of precedence (following BOMDAS). Each level is represented by a non-terminal, and its position in the grammar determines its precedence (lower has higher precedence since they're expanded first during parsing).

S (Top) lowest precedence	for addition and subtraction
A	for multiplication and division
B	for left and right parentheses (brackets)
C (Bottom) highest precedence	for exponents

At the lowest level (variable S), addition and subtraction are handled. It was structured in a way that forces the right operand to be at least a term, ensuring that addition and subtraction are evaluated after everything else, enforcing the fact that they have the least precedence. Additionally, this level recurses on the left, forcing the left associativity, which is standard for addition and subtraction.

At level 2 (variable A), multiplication and division are handled. A's position in the hierarchy is intentional for the precedence of these operations and ensures that multiplication and division are evaluated before addition and subtraction, which are at level 1 (i.e. variable S), but not before brackets and exponents (variables C and B, respectively). Additionally, this also recurses on the left, enforcing left associativity of multiplication and division.

At level 3 (variable B), exponents are handled. This position in the hierarchy ensures that exponents have a higher precedence than addition, subtraction, multiplication and division, but lower precedence than left and right parentheses (brackets). Unlike the other levels, level 3 recurses to the right since exponents have right associativity.

At level 4 (variable C), brackets/ parentheses, variables and numbers are handled. Its position is intentional as it gives this level the highest precedence. It also accounts for atomic units in any arithmetic expression. "Number" is responsible for any literal number that can be used in the expression, while "variable" accounts for any variable name used in the expression that represents a number.

Ambiguity Analysis

Ambiguity

This grammar is not ambiguous, as it only allows for one parse tree that enforces BOMDAS (arithmetic operation precedence). It does this by following a strict hierarchy, putting every operator at exactly one level, which in turn forces the parser to group by level. Additionally, with how each rule is structured, the program ensures left recursion for addition, subtraction, multiplication and division, and right recursion for exponents, which also eliminates ambiguity regarding their associativity.

How does ambiguity affect parsing?

A parser should come to a single decision. If the grammar is ambiguous, the parser will eventually reach a point where multiple rules can be applied to the same input, yielding different results, and it would have no way to decide which is correct. Having multiple rules that can be applied makes the parser non-deterministic, which means it will have to explore all possible parse trees. This would make the program slower and impractical. Finally, if the grammar is ambiguous, it becomes harder to identify errors. Exploring multiple rules means that a path can

report an error in the tree, when that same error might not be present/ apply to another rule; this makes error messages unreliable and misleading.

Parsing Approach and Algorithm

The syntax checker we implemented uses a recursive descent parse which starts from the start symbol (variable S) and uses a set of recursive functions where each function responds to a variable or non-terminal in the grammar. The parser enforces the associativity of the operators by using loops in the parseS() and parseA() functions for left associativity and using recursion in the parseB() for right associativity.

The parsing algorithm:

1. The input string is first processed by the lexer where it converts the input into a list of tokens.
2. The parser starts with the start symbol by calling its parse function.
3. The parse function for the start symbol processes addition and subtraction which have the least precedence of all the operations. It first parses a term by using the parse function for the variable A and then it checks for more additions and subtractions in the expression and continuous parsing them.
4. The parse function for variable A processes multiplication and division in the arithmetic expression and calls the parse function for variable B and handles at additional multiplication and division in the expression.
5. The parse function for variable B processes exponentiation so if an exponent is found, it recursively parses towards the right hand side since it has right associativity.
6. The parse function for variable C handles parentheses in the arithmetic expression, as well as, numbers and variables.
7. Where unexpected tokens are found, the parser displays a syntax error.
8. And after parsing, the parser ensures all input is consumed (that is the entire arithmetic expression) and if extra tokens remain the input is rejected.

So if parsing is successful then the expression is valid, but if parsing fails then an error message is displayed and a parse tree is generated for valid inputs.

CNF Conversion Steps

Final Grammar :
S → S + A | S - A | A
A → A * B | A / B | B
B → C ^ B | C
C → (S) | variable | number

1. New Start state
   N → S
   S → S + A | S - A | A
   A → A * B | A / B | B
   B → C ^ B | C
   C → (S) | variable | number

2. Remove unit productions
   N → S + A | S - A | A * B | A / B | C ^ B | (S) | variable | number
   S → S + A | S - A | A * B | A / B | C ^ B | (S) | variable | number
   A → A * B | A / B | C ^ B | (S) | variable | number
   B → C ^ B | (S) | variable | number
   C → (S) | variable | number

3. Convert to binary rules (the right hand side should have no more than 2 variables)

   So, the final Chomsky Normal Form:

   N → SK | SL | AM | AO | CP | IQ | variable | number
   S → SK | SL | AM | AO | CP | IQ | variable | number
   A → AM | AO | CP | IQ | variable | number
   B → CP | IQ | variable | number
   C → IQ | variable | number
   D → +
   E → -
   F → *
   G → /
   H → ^
   I → (
   J →)
   K → DA
   L → EA

M → FB
O → GB
P → HB
Q → SJ

Parse Trees or Derivations for Sample Output

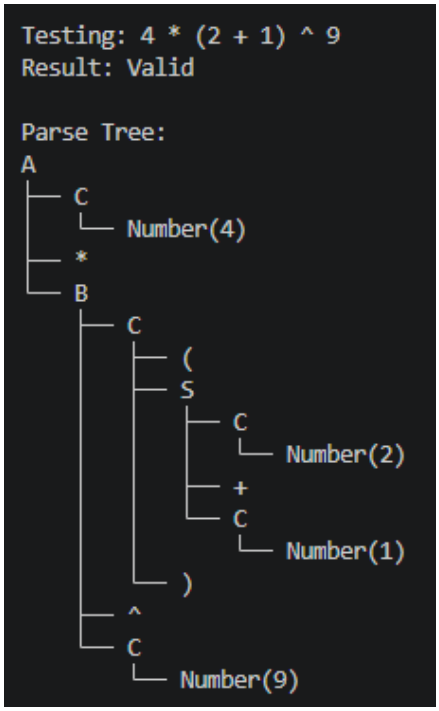
Valid Test Cases

```
Testing: 5 - 4
Result: Valid

Parse Tree:
S
├── C
│   └── Number(5)
├── -
└── C
    └── Number(4)
```

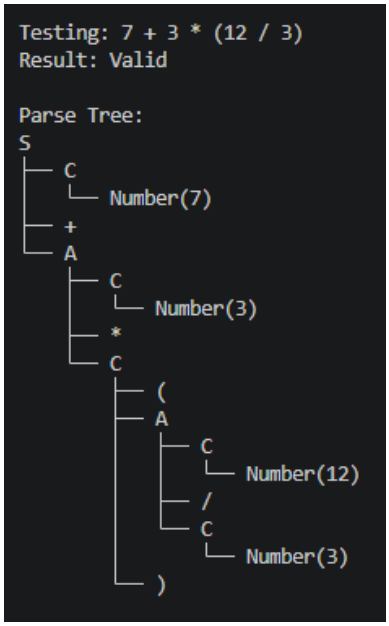
For this test case, a simple subtraction was done. Subtraction falls in the first level of the grammar (S), hence S is at the root of the tree. The next level of the tree is occupied by the lowest level of the grammar (C), this level is responsible for parentheses, numbers and variables. It contains the numbers in this case, as leaf nodes.

```
Testing: 3 * (2 - 1)
Result: Valid

Parse Tree:
A
├── C
│   └── Number(3)
├── *
└── C
    ├── (
    ├── S
    │   ├── C
    │   │   └── Number(2)
    │   ├── -
    │   └── C
    │       └── Number(1)
    └── )
```

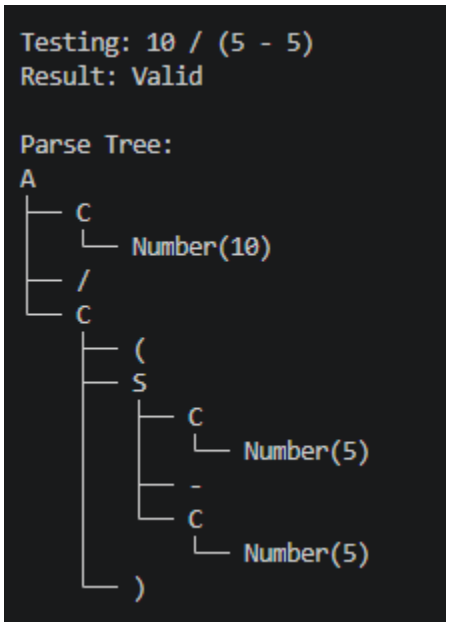
For this test case, at the leaf nodes is the part of the expression within the parentheses. It is at the lowest level of the tree because it has the highest precedence.



This test case shows that the part of the expression within the parentheses is at the lowest level of the tree even though it is in the middle of the entire expression. This is because it has the highest precedence, followed by the exponent (^ 9) and then the multiplication.



Another valid test case showing precedence and how the different levels are structured within the tree.



Another valid test case.

No parse trees are generated from invalid expressions.

Invalid Test Cases

```
Testing: 5(7 - 8)
Result: Invalid
Error: Syntax error: unexpected token '('
```

This test case shows a minor limitation of the syntax checker. Here it identifies this expression as invalid because no valid arithmetic operator was placed before the first parenthesis.

```
Testing: 3*-(2 + 1)
Result: Invalid
Error: Syntax error: expected a number, variable, or '(' but found '-'
```

This expression was marked as invalid because it has two operators placed beside each other.

```
Testing: 6 * (8 + 5) /
Result: Invalid
Error: Syntax error: expected a number, variable, or '(' but found ''
```

This expression was invalid because it had a division operator but no number or variable to follow.

```
Testing: 4 + * 3
Result: Invalid
Error: Syntax error: expected a number, variable, or '(' but found '*'
```

Again, here we have chained operators resulting in an invalid expression.

```
Testing: (2 + 3
Result: Invalid
Error: Syntax error: expected RBRACKET but found ''
```

For this expression, no closing parenthesis was added to the expression, making it invalid.

Discussion of Correctness and Limitations

Correctness

The parser is correct because it follows the grammar striclty so each function implements a production rule and only valid expressions are accepted. Operator precedence is enforced as the grammar ensures exponents are evaluated first, then parentheses/ brackets, then multiplication and division, and lastly, addition and subtraction based on BOMDAS; associativity is also correctly processed, with left associativity for addition, subtraction, multiplication, and division, and right associativity for exponentiation. The parser also checks that all tokens are consumed and extra or missing tokens result in an error.

Limitations

The limitations of the parser are as follows:

1. There's no semantic validation as the parser only checks syntax, it does not calculate the result of the arithmetic expressions nor does it detect errors like division by zero.
2. The parser only supports simple arithmetic expressions but not complex programming concepts or constructs, like functions and assignments.
3. The error messages indicate unexpected tokens but they do not provide suggestions for correction in the expression input.
4. The parser does not support implicit multiplication so expressions such as 4(5 + 6) are mathematically valid, but the parser requires an explicit multiplication operator (*) before the left parenthesis without it the input is treated as a syntax error, as seen in the test case below: